

UNIVERSIDAD NACIONAL DE GENERAL SARMIENTO

Sistemas Operativos y Redes II - Código A0533

TRABAJO PRÁCTICO 2

Seguridad en Sistemas Operativos

Hardening guiado - AppArmor - auditd - AIDE - strace

1. Propósito del trabajo

Este trabajo propone proteger y observar una máquina GNU/Linux desde tres perspectivas que se complementan: reducir configuraciones innecesariamente riesgosas, limitar lo que puede hacer una aplicación y conservar evidencia de los cambios que ocurren. El recorrido es guiado porque gran parte del aprendizaje se realizará fuera del aula, pero cada grupo deberá completar configuraciones, interpretar resultados y justificar sus decisiones.

El TP no busca que memoricen comandos ni que apliquen un benchmark completo. Busca que puedan explicar qué riesgo reduce cada control, cómo comprobaron su efecto y qué evidencia produciría un incidente

2. Alcance y relación con la materia

U2: relación entre aplicaciones, llamadas al sistema, kernel y herramientas de observación como strace.
U3: mínimo privilegio, hardening, control de acceso obligatorio, auditoría persistente e integridad.

3. Entorno obligatorio y seguridad

Elemento	Requisito
Sistema	Ubuntu Server 24.04 LTS amd64.
Modalidad	Máquina virtual aislada. No usar un equipo personal ni un servidor real.
Snapshot	Crear TP2_BASE antes de aplicar cambios. Es parte obligatoria de la evidencia.
Acceso	Mantener acceso por consola de la VM antes de cambiar SSH.
Herramientas	AppArmor, auditd, AIDE, strace, gcc, make y OpenSSH Server.
Datos	Solo archivos ficticios dentro de /srv/tp2. No trabajar con información real.

La ruta oficial está diseñada y documentada para Ubuntu Server 24.04 LTS. No se aceptarán cambios de distribución sin autorización previa, porque las rutas, servicios y mecanismos de seguridad no son idénticos.

4. Archivos provistos

```
codigo_base/
├── Makefile
├── src/
│   ├── tp2_reader.c
│   └── tp2_event.c
├── hardening/hardening_base.sh
├── apparmor/usr.local.bin.tp2-reader.base
├── audit/99-tp2.rules.base
├── aide/aide.conf.base
├── scripts/
│   ├── preparar_entorno.sh
│   ├── escenario_final.sh
│   └── restaurar_entorno.sh
└── tests/public_checks.sh
```

Los programas tp2-reader y tp2-event son aplicaciones controladas de laboratorio. No contienen una vulnerabilidad ni realizan acciones ofensivas. Se utilizan para generar operaciones reproducibles que luego serán observadas con AppArmor, strace, auditd y AIDE.

5. Secuencia de trabajo

1. Leer la consigna completa y preparar la VM.
2. Ejecutar el preparador, verificar las herramientas y crear el snapshot TP2_BASE.
3. Completar y validar el script de hardening.
4. Completar el perfil AppArmor y demostrar la diferencia entre complain y enforce.
5. Completar las reglas auditd y la configuración AIDE.
6. Correlacionar una ejecución observada por strace con el evento persistente de auditd.
7. Ejecutar el escenario integrador, reunir evidencia y redactar el informe.

6. Parte 0 - Preparación del entorno

Desde la raíz del proyecto, ejecute:

```
sudo ./scripts/preparar_entorno.sh --install-packages
```

El script instala las dependencias, compila las dos aplicaciones, crea el usuario de laboratorio y prepara /srv/tp2. Al finalizar, guarde la salida y cree el snapshot TP2_BASE.

Verifique el entorno:

```
sudo ./tests/public_checks.sh --entorno
```

El script admite tres modos: --estructura verifica el material provisto y la compilación; --entorno agrega la verificación de la máquina virtual y requiere sudo; --entrega agrega la verificación de los cuatro entregables completos. Los archivos terminados en .base y hardening_base.sh son plantillas de la cátedra: conservan sus bloques TODO de forma intencional y no deben modificarse ni eliminarse.

No continúe si...

AppArmor no está cargado, auditctl no puede consultar el kernel, falta alguna herramienta o no se creó el snapshot. Resolver el entorno al comienzo evita repetir todo el trabajo más adelante.

7. Parte 1 - Hardening guiado

Complete `hardening/hardening_base.sh` y guárdelo como `hardening/hardening.sh`. El marco del script ya incluye registro, contadores, respaldos y modos de ejecución. Su tarea es implementar cuatro controles modificables y conservar el control de diagnóstico de UID 0.

Control	Qué debe lograr	Cómo se valida
UID 0	Detectar cuentas con UID 0 distintas de root. No corregir automáticamente.	La salida debe identificar una desviación o confirmar que solo root tiene UID 0.
OpenSSH	Crear un fragmento separado que fije <code>PermitRootLogin no</code> .	Validar con <code>sshd -t</code> antes de recargar y comprobar la configuración efectiva.
Calidad de contraseñas	Configurar una política acotada en <code>pwquality.conf.d</code> .	Confirmar que <code>pam_pwquality</code> está activo y que el archivo coincide con la política definida.
Umask	Configurar <code>umask 027</code> para nuevas sesiones mediante <code>profile.d</code> .	Abrir una nueva sesión y verificar la máscara y los permisos de un archivo nuevo.
Enlaces protegidos	Persistir <code>fs.protected_hardlinks=1</code> y <code>fs.protected_symlinks=1</code> .	Comprobar el archivo en <code>sysctl.d</code> y los valores efectivos del kernel.

El script debe aceptar:

```
sudo ./hardening/hardening.sh --check
sudo ./hardening/hardening.sh --apply
sudo ./hardening/hardening.sh --restore
```

- Primera ejecución de `--apply`: aplica solamente lo que falta.
- Segunda ejecución de `--apply`: no repite cambios y registra `SKIPPED`.
- `--check`: informa el estado sin modificar.
- `--restore`: repone respaldos o elimina fragmentos que no existían antes.
- Toda acción queda en `/var/log/tp2-hardening.log` con fecha y estado.

Evidencia mínima de la Parte 1

- Salida resumida de `--check` antes del `hardening`.
- Primera y segunda ejecución de `--apply`.
- Fragmentos creados y validaciones realizadas.
- Demostración de que `--restore` funciona o explicación de por qué se usará el snapshot para la restauración final.
- Análisis 1: elegir un control y explicar el riesgo que reduce y cómo se comprobó su efecto.
- Análisis 2: demostrar, usando las dos ejecuciones, por qué el script es idempotente.

8. Parte 2 - Control obligatorio con AppArmor

Los archivos `publico.txt` y `confidencial.txt` tienen permisos DAC de lectura. Antes de cargar un perfil, `tp2-reader` puede leer ambos. El objetivo es crear un perfil que permita el recurso público pero no conceda permiso para el recurso confidencial.

```
/usr/local/bin/tp2-reader /srv/tp2/datos/publico.txt
/usr/local/bin/tp2-reader /srv/tp2/datos/confidencial.txt
```

Complete el archivo base y cópielo a `/etc/apparmor.d/usr.local.bin.tp2-reader`. Luego siga esta secuencia:

1. Validar la sintaxis con `apparmor_parser` antes de cargar el perfil.
2. Cargar el perfil y colocarlo en modo `complain`.
3. Leer ambos archivos y localizar el evento en el journal del kernel.

4. Cambiar a enforce.
5. Comprobar que publico.txt se lee y confidencial.txt es rechazado.
6. Guardar el mensaje de denegación y explicar qué regla faltaba.

```
sudo apparmor_parser -Q /etc/apparmor.d/usr.local.bin.tp2-reader
sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.tp2-reader
sudo aa-complain /usr/local/bin/tp2-reader
sudo aa-enforce /usr/local/bin/tp2-reader
sudo journalctl -k --since "-10 minutes" --no-pager | grep -i apparmor
```

DAC responde si el usuario y sus grupos pueden acceder al archivo. AppArmor agrega una decisión basada en el perfil de la aplicación. Para demostrar MAC, primero deben probar que DAC sí permitía la lectura

Evidencia mínima de la Parte 2

- Permisos y propietarios de los dos archivos.
- Lectura exitosa sin perfil.
- Ejecución en complain con evento registrado.
- Ejecución en enforce: acceso público permitido y acceso confidencial bloqueado.
- Análisis 3: demostrar con evidencia que DAC permitía y AppArmor bloqueó.
- Análisis 4: explicar la diferencia práctica entre complain y enforce usando el caso realizado.

9. Parte 3 - Auditoría, syscalls e integridad

9.1 Reglas auditd

Complete exactamente dos reglas en audit/99-tp2.rules y cópielas a /etc/audit/rules.d/99-tp2.rules: una regla para modificaciones dentro de /srv/tp2/datos y una regla para la ejecución de tp2-event. Cargue y liste las reglas:

```
sudo augenrules --load
sudo auditctl -l | grep tp2
```

9.2 Correlación entre strace y auditd

Ejecute el mismo programa bajo strace y búsquelo luego en auditd. Debe relacionar al menos timestamp, PID, ejecutable, syscall y resultado.

```
sudo strace -f -e trace=execve,openat,write -o /srv/tp2/evidencias/strace_tp2_event.txt /usr/local/bin/tp2-event
"evento_grupo_N"

sudo ausearch -k tp2_exec -ts recent -i
sudo ausearch -k tp2_datos -ts recent -i
```

9.3 Línea base con AIDE

Complete aide/aide.conf y use esa configuración local. No utilice la configuración global para recorrer todo el sistema.

```
sudo install -m 0644 aide/aide.conf /srv/tp2/config/aide.conf
sudo aide --config=/srv/tp2/config/aide.conf --config-check
sudo aide --config=/srv/tp2/config/aide.conf --init
sudo mv /srv/tp2/aide/aide.db.new /srv/tp2/aide/aide.db
```

Después de crear la base, realice tres cambios controlados: crear un archivo, modificar el contenido de otro y cambiar sus permisos. Ejecute AIDE y explique dónde aparece cada diferencia.

```
sudo aide --config=/srv/tp2/config/aide.conf --check
```

Evidencia mínima de la Parte 3

- Archivo de reglas y salida de auditctl -l.
- Fragmento relevante de strace y evento correspondiente de ausearch.
- Configuración local de AIDE y confirmación de la línea base.
- Reporte de AIDE con archivo nuevo, contenido modificado y permisos cambiados.
- Análisis 5: correlacionar una syscall de strace con el evento persistente de auditd.
- Análisis 6: explicar qué detectó AIDE y una limitación de una base almacenada en la misma máquina.

10. Escenario integrador

Con AppArmor en enforce, las reglas auditd cargadas y la base AIDE activa, ejecute:

```
sudo ./scripts/escenario_final.sh
```

El grupo deberá reconstruir lo ocurrido: qué operación fue permitida, cuál fue bloqueada, qué evento quedó en auditd y qué cambio detectó AIDE. No se pide un nuevo cuestionario: esta explicación forma parte de la conclusión del informe.

11. Entrega

```
tp2-grupo-N/  
├── README.md  
├── hardening/hardening.sh  
├── apparmor/usr.local.bin.tp2-reader  
├── audit/99-tp2.rules  
├── aide/aide.conf  
├── evidencias/  
└── informe.pdf
```

Antes de comprimir la carpeta, ejecute `./tests/public_checks.sh --entrega desde codigo_base`. Ese modo confirma que los cuatro entregables existen, que `hardening.sh` tiene sintaxis válida y que no quedan bloques TODO sin resolver.

Elemento	Contenido esperado
README.md	Integrantes, entorno, orden de ejecución y problemas relevantes.
Informe	8 a 12 páginas de contenido, sin copiar salidas completas ni convertirlo en tutorial.
Evidencias	Archivos de texto y capturas indispensables, organizados por parte.
Código/configuración	Versiones finales utilizadas y verificadas.
Repositorio/ZIP	Sin claves, imágenes de VM, bases de contraseñas ni datos personales.

En el informe, cada evidencia debe indicar qué se intentó, qué resultado se esperaba, qué ocurrió y qué demuestra. Una captura sin interpretación no constituye evidencia suficiente.

15. Referencias técnicas

- [Ubuntu Server - AppArmor](#)
- [Ubuntu Server - OpenSSH Server](#)
- [Ubuntu manpages - auditctl\(8\)](#)
- [Ubuntu manpages - audit.rules\(7\)](#)
- [Ubuntu manpages - aide\(1\)](#)
- [AIDE - documentación del proyecto](#)